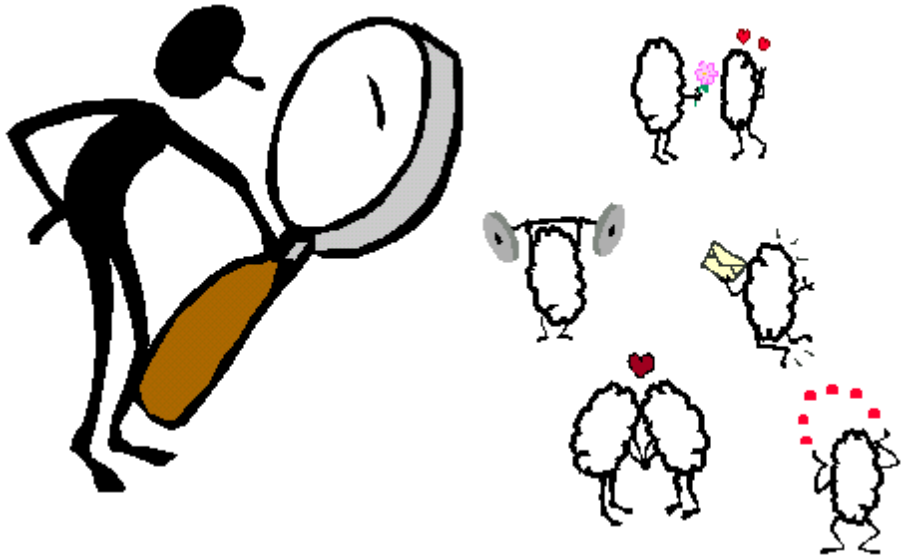
Chapter 3 Object Interactions



Contents



- Java Methods Overview
- “this” Keyword
- Method Overload
- Methods
- Message Passing Between Objects
- Information Hiding/Accessibility

3.1 Java Methods Overview



Class definitions

- **Classes** describe **objects** of the same type in general
- **State** (attributes, aka fields)
- **Behavior** (methods)
  - Including constructors

BankAccount.java

```
public class BankAccount
{
  private int balance;                    } attributes (aka fields)

  public BankAccount()                    } constructor
  {
    this.balance = 0;
  }

  public int getBalance()                 } methods
  {
    return this.balance;
  }

  public void deposit( int amount )
  {
    int newBalance = this.balance + amount;
    this.balance = newBalance;
  }
  ...
}
```

Attributes

- **Variables** for storing information that an object has
- **Defined** inside the class body, but outside of any methods
- **Written** in Java as
  - <visibility> <type> <name>

BankAccount.java

```
public class BankAccount
{
  private int balance;

  public BankAccount(int newBalance)
  {
    this.balance = newBalance;
  }

  public int getBalance()
  {
    return this.balance;
  }

  public void deposit( int amount )
  {
    int newBalance = this.balance + amount;
    this.balance = newBalance;
  }
  …
}
```

attributes (aka fields)

constructor

methods

Methods

- **Methods have a *signature* and *a body* of statements (in braces)**
- **Can have *parameters***
- **Can have *a return value***
- **Can have *local variables***

BankAccount.java

```
public class BankAccount
{
  private int balance;

  public BankAccount()
  {
    this.balance = 0;
  }

  public int getBalance()
  {
    return this.balance;
  }

  public void deposit( int amount )
  {
    int newBalance = this.balance + amount;
    this.balance = newBalance;
  }
  …
}
```

attributes (aka fields)

constructor

methods

Invoking Methods

- To call a method, use the **dot operator**
  - The same operator is used for both class and instance methods
  - If the call is to a method of the same class, the dot operator is not necessary

```
BankAccount account = new BankAccount();
account.setOwner("Smith");
account.credit(1000.0);
System.out.println(account.getBalance());
...
```

BankAccount method

```
public void credit(double amount) {
  setBalance(getBalance() + amount);
}
```

How do you USE objects?

```
public class BankDemo1
{
    public void execute()
    {
        BankAccount bobAcct;
        bobAcct = new BankAccount();

        int balance1 = bobAcct.getBalance();

        System.out.println( "Bob's balance is now: " + balance1 );

        System.out.println( "Depositing 50 to Bob's

        bobAcct.deposit( 50 );

        System.out.println( "Bob's balance is now: " + bobAcct.getBalance() );
    }
}
```

DECLARE a variable that can "point" to a BankAccount

CREATE a new BankAccount and make account point to it

CALL getBalance() method of account. Returns an int

Call deposit method with amount as argument.

CALL getBalance() method again to get the balance again

How Methods Work

BankProgram.java

```
…
int aliceBalance =
aliceAccount.getBalance();
…
```

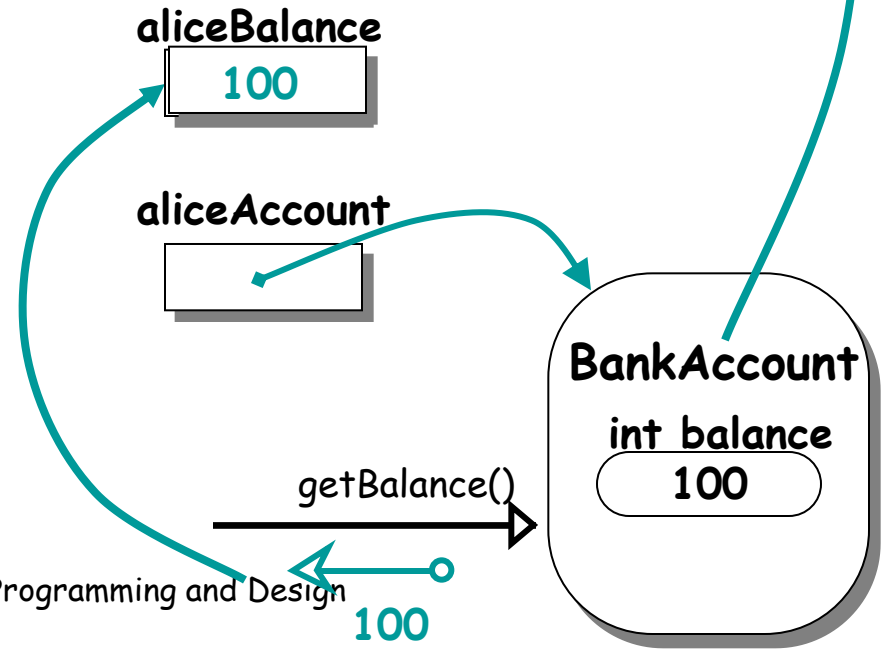
BankAccount.java

```
public int getBalance()
{
   return balance;
}
```

When the line above is run …

1) Find object pointed to by aliceAccount
2) Find code for class of that object
3) Find code for getBalance()
4) Run code
5) Return value
6) Use returned value

State of Memory



Passing Primitive Types

Code

```
x = 10;
y = 20;
tester.myMethod( x, y );
```

A

```
public void myMethod( int one, float
two )
{
    one = 25;
    two = 35.4f;
}
```

State of Memory

At A before myMethod

x 10

y 20

A. Local variables do not exist before the method execution

Passing Primitive Types

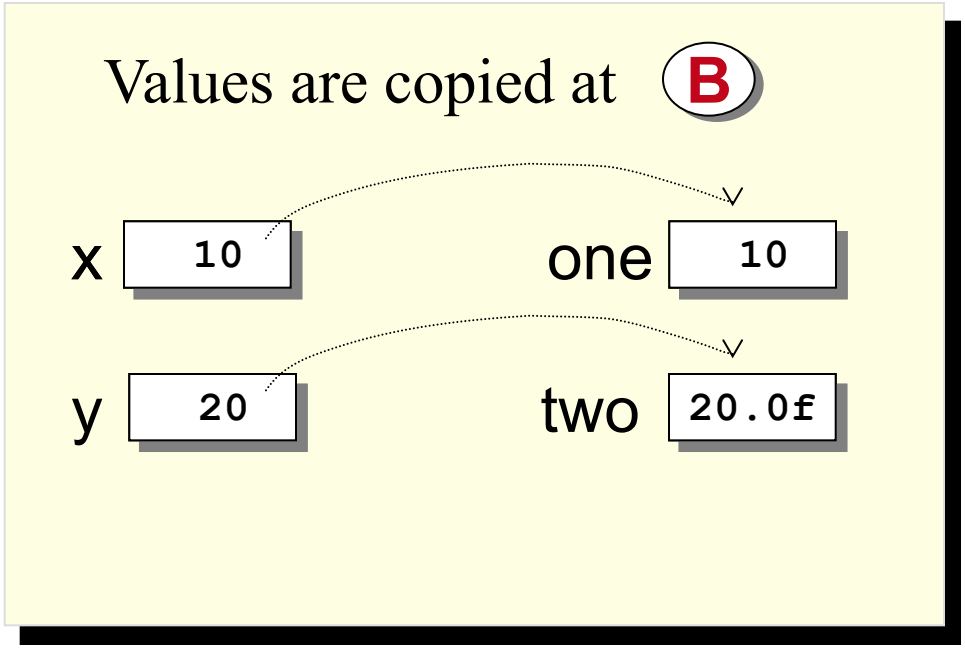
Code

```
x = 10;
y = 20;
tester.myMethod( x, y );
```

```
public void myMethod( int one, float
two )
{
    one = 25;
    two = 35.4f;
}
```

B

State of Memory



B. The values of arguments are copied to the parameters.

Passing Primitive Types

Code

```
x = 10;
y = 20;
tester.myMethod( x, y );
```

```
public void myMethod( int one, float
two )
{
    one = 25;
    two = 35.4f;
}
```

C

State of Memory

After C is executed

x	10	one	25
y	20	two	35.4f

C. The values of parameters are changed.

Passing Primitive Types

Code

```
x = 10;
y = 20;
tester.myMethod( x, y );
```

D

```
public void myMethod( int one, float
two )
{
    one = 25;
    two = 35.4f;
}
```

State of Memory

At D after myMethod

x 10

y 20

D. Parameters are erased. Arguments remain unchanged.

Passing Parameters

- **Arguments** are passed to a method using the ***pass-by-value*** scheme
  - Parameters and arguments ***do not*** have to have the same name
  - Whether or not they have the same name, parameters are separate ***copies*** of the arguments
  - Parameters are ***local*** to the method, i.e., they only exist while inside the method. Changes made to the parameters will not affect the value of corresponding arguments

*** Note: the argument is the variable or value given in the method *call*, while the parameter is the variable in the method *definition* that copies that value)**

Passing Objects as Parameters

- **Object-type** parameters copy the ***reference*** to the passed object
  - calls to methods of the parameter ***does*** affect the original object's state
  - sometimes known as **"pass-by-reference"**
  - BUT, if we assign parameter variable to another object, then original object is not affected anymore
  - Think of it this way:
    - ***pass-by-value***, but the value being passed is the ***pointer*** to the object, not the object itself

Passing Objects

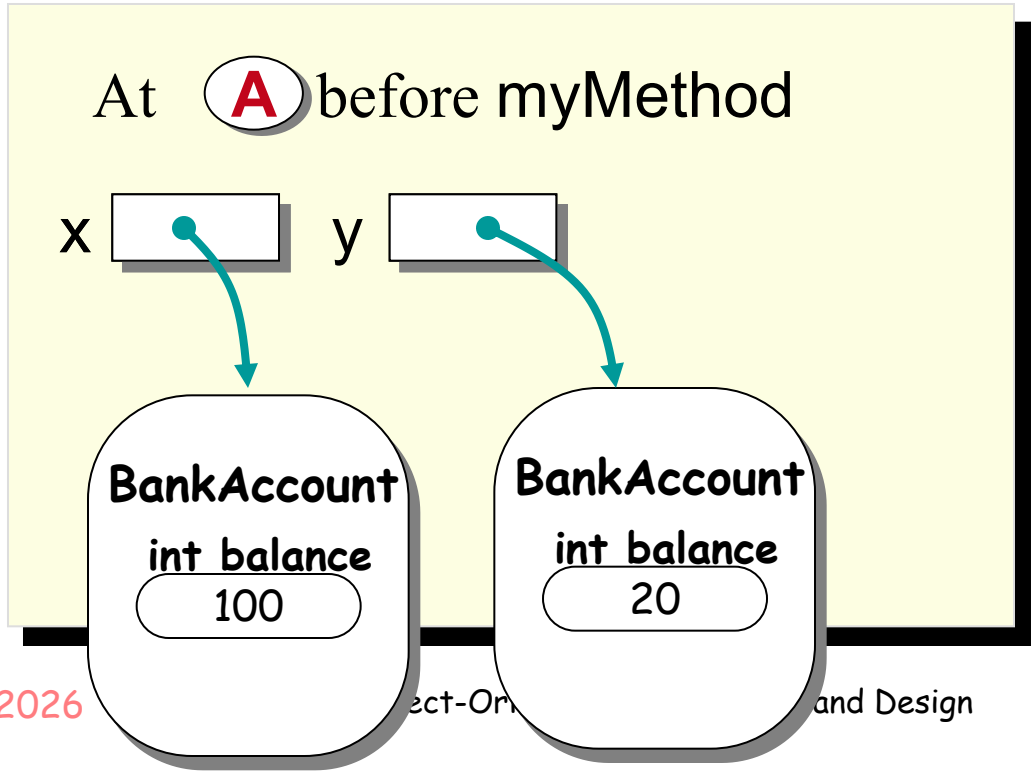
in BankApplet

```
BankAccount x =
        new BankAccount( 100 );
BankAccount y =
        new BankAccount( 20 );
tester.myMethod( x, y );
```

A

```
public void myMethod( BankAccount one,
                      BankAccount two )
{
    one.deposit( 50 );
    two.withdraw( 10 );
}
```

State of Memory



A. Local variables do not exist before the method execution

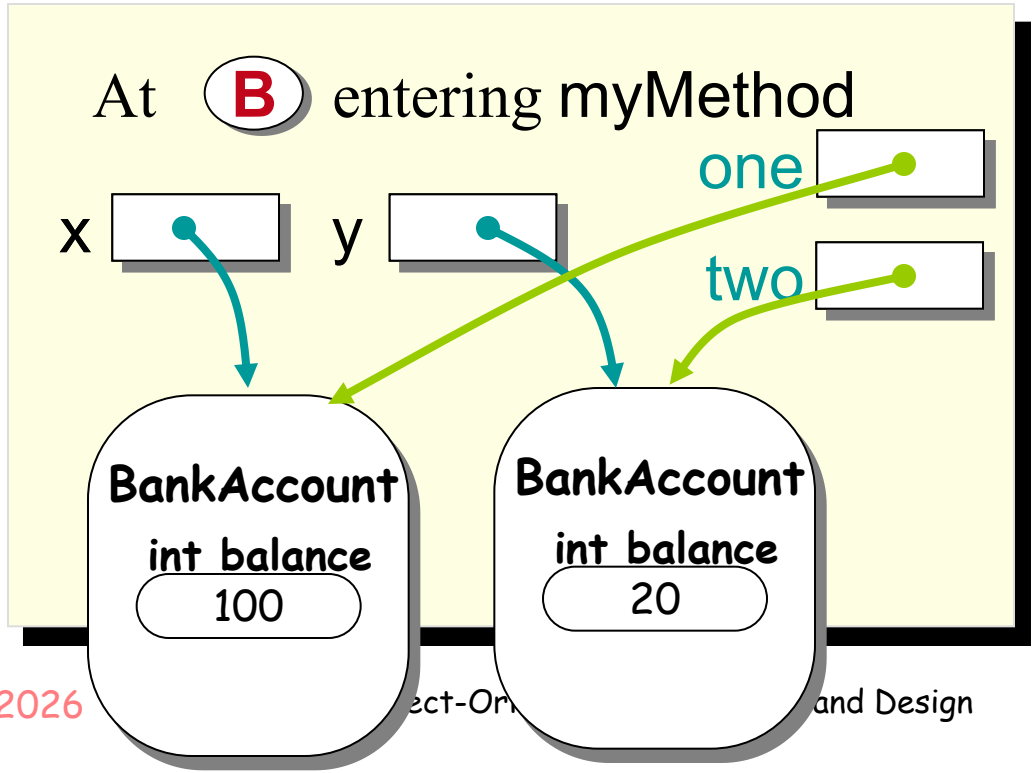
Passing Objects

in BankApplet

```
BankAccount x =
        new BankAccount( 100 );
BankAccount y =
        new BankAccount( 20 );
tester.myMethod( x, y );
```

```
public void myMethod( BankAccount one,
                      BankAccount two )
{
    one.deposit( 50 );
    two.withdraw( 10 );
}
```

State of Memory



B. Parameters **one** and **two** copy the *references* to the objects pointed to by **x** and **y**

Passing Objects

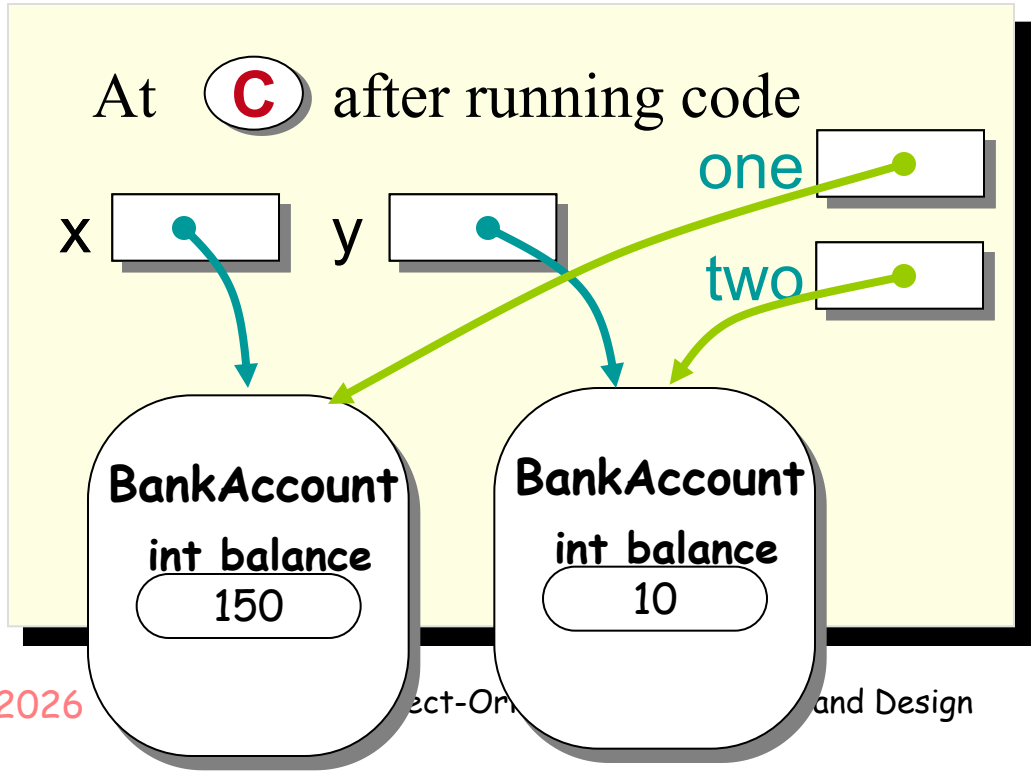
in BankApplet

```
BankAccount x =
        new BankAccount( 100 );
BankAccount y =
        new BankAccount( 20 );
tester.myMethod( x, y );
```

```
public void myMethod( BankAccount one,
                      BankAccount two )
{
    one.deposit( 50 );
    two.withdraw( 10 );
}
```

C

State of Memory



C. Method calls to **one** and **two** affect the same objects referred to by **x** and **y**

Passing Objects

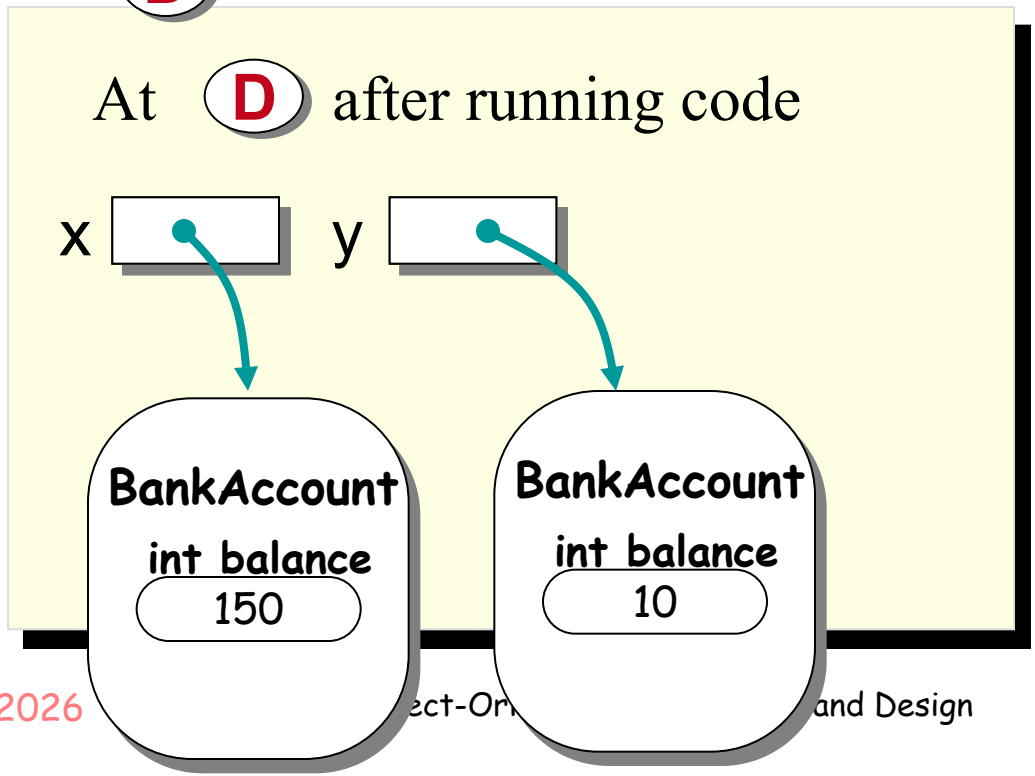
in BankApplet

```
BankAccount x =
        new BankAccount( 100 );
BankAccount y =
        new BankAccount( 20 );
tester.myMethod( x, y );
```

```
public void myMethod( BankAccount one,
                      BankAccount two )
{
    one.deposit( 50 );
    two.withdraw( 10 );
}
```

D

State of Memory



D. When myMethod returns, parameters **one** and **two** disappear. But objects **x** and **y** have been changed.

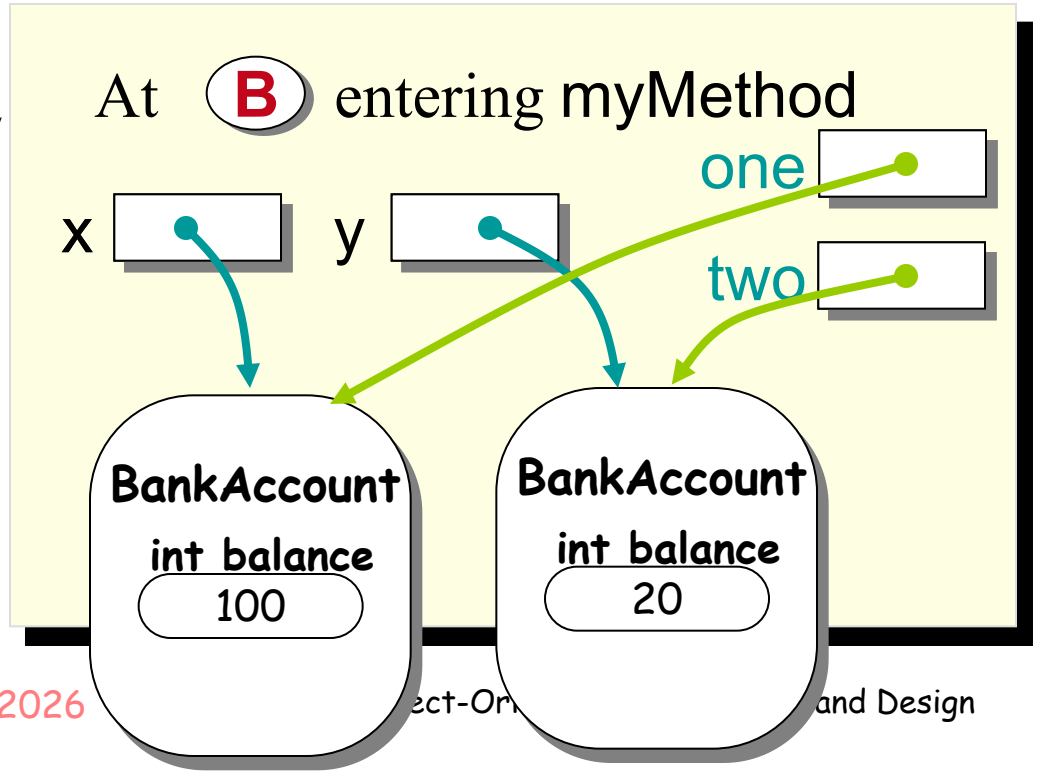
Reassigning Parameters

in BankApplet

```
BankAccount x =
        new BankAccount( 100 );
BankAccount y =
        new BankAccount( 20 );
tester.myMethod( x, y );
```

```
public void myMethod( BankAccount one,
                      BankAccount two )
{
    one = two;
    one.deposit( 50 );
    two.withdraw( 10 );
}
```

State of Memory



Suppose we reassign one to something else within the method body

Reassigning Parameters

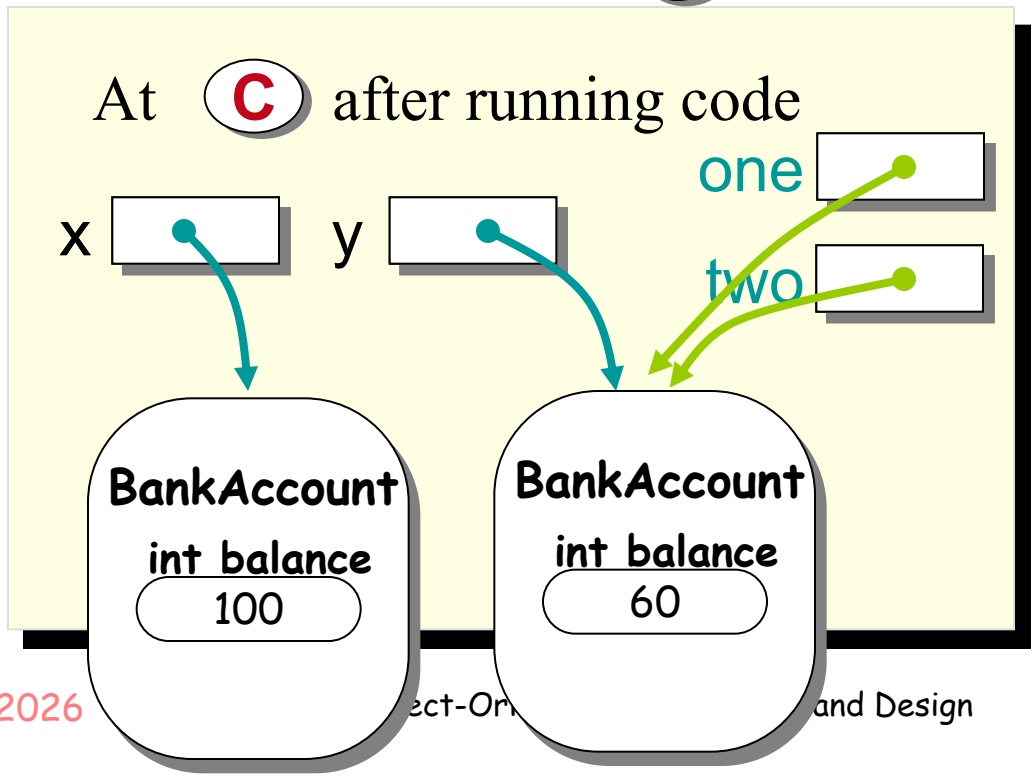
in BankApplet

```
BankAccount x =
        new BankAccount( 100 );
BankAccount y =
        new BankAccount( 20 );
tester.myMethod( x, y );
```

```
public void myMethod( BankAccount one,
                      BankAccount two )
{
    one = two;
    one.deposit( 50 );
    two.withdraw( 10 );
}
```

C

State of Memory



C. Method calls to **one** and **two** affect the *same* object (same as pointed to by y)
x is ***not*** affected.

Passing Objects

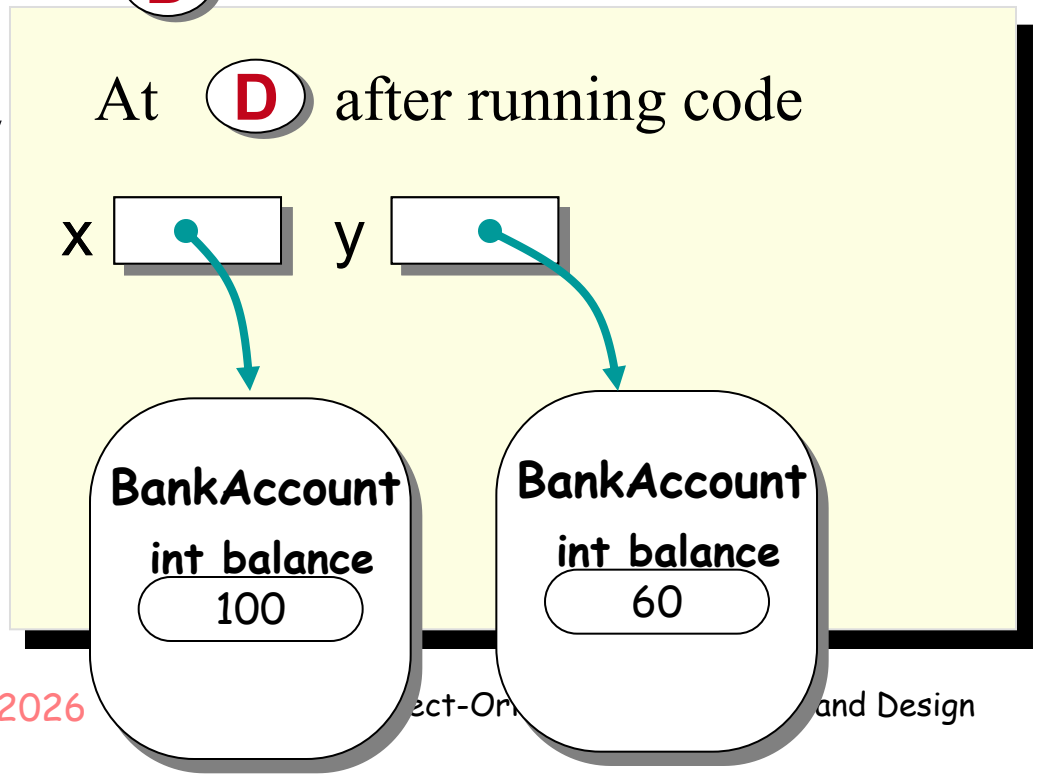
in BankApplet

```
BankAccount x =
        new BankAccount( 100 );
BankAccount y =
        new BankAccount( 20 );
tester.myMethod( x, y );
```

D

```
public void myMethod( BankAccount one,
                      BankAccount two )
{
    one = two;
    one.deposit( 50 );
    two.withdraw( 10 );
}
```

State of Memory



D. When myMethod returns, parameters **one** and **two** disappear. Objects **y** has been changed.

A Common Error

```
public class BankDemo1{
   public void execute()    {
      BankAccount bobAcct;

      int balance1 = bobAcct.getBala
      System.out.println( "Bob's bala
   balance1 );
      System.out.println( "Depositing 50 to Bob's
   account ..." );

      bobAcct.deposit( 50 );
      System.out.println( "Bob's balance is now: "
         + bobAcct.getBalance() );
   }
}
```

Forgot to create a new BankAccount and assign it to account

What will go wrong?

A Common Error

```
public class BankDemo1
{
   public void execute()
   {
      BankAccount bobAcct;

      int balance1 = bobAcct.getBalance();

      System.out.println( "Bob's balance is now: " + balance1 );

      System.out.println( "Depositing 50 to Bob's account ..." );

      bobAcct.deposit( 50 );

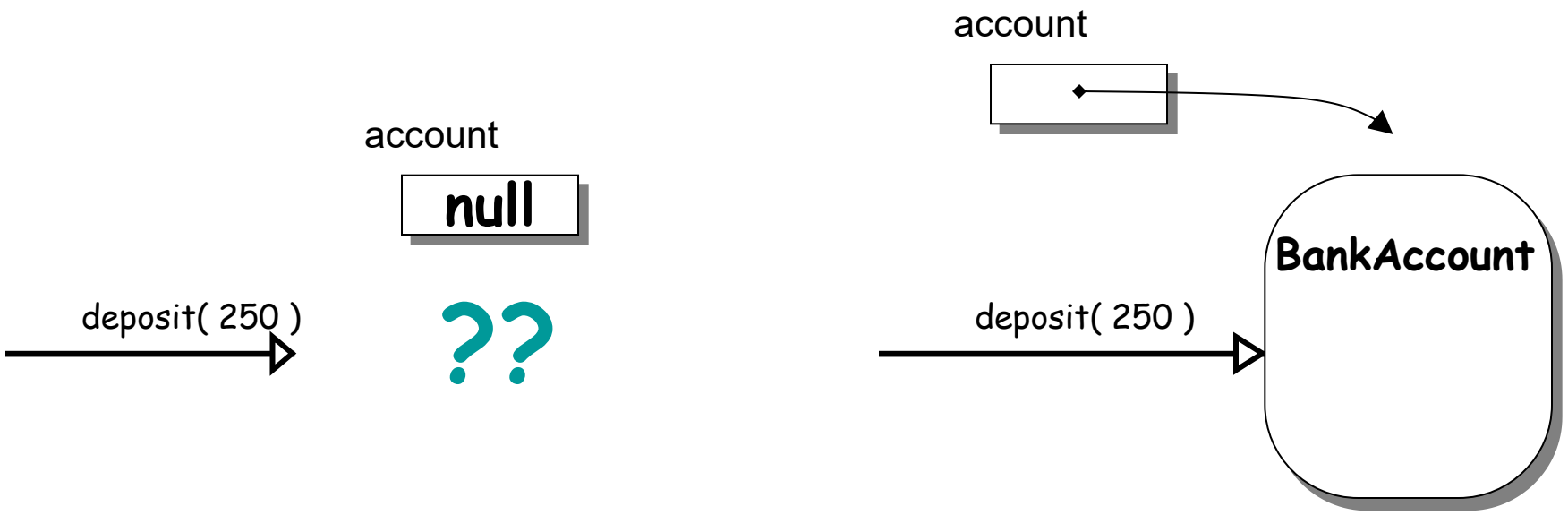
      System.out.println( "Bob's balance is
         + bobAcct.getBalance() );
   }
}
```

Forgot to create a new BankAccount and assign it to account

You will get a NullPointerException!

Calling a Method on a Null Pointer

- **NullPointerException:** when you try to call a method or access a field on an object variable that is not pointing to anything
- **Remember that you need to create a new object before using it!**



Points to Remember

- **Remember: Object variables are references (aka pointers)**
  - Point to ***"null"*** by default
  - Need to call ***new*** to create a new object
  - Different variables can point to same object at the same time
  - If no pointers point to an object, then it gets collected by the "garbage collector"
    - You can't access it anyway if you don't have a pointer to it
    - This allows Java to automatically save on memory

3.2 “this” keyword



Example: Two Accounts

```
public class BankAccount
{
  private int balance;

  public void deposit( int amount )
  {
    balance = balance + amount;
  }
  …
}
```

```
BankAccount bobAcct;

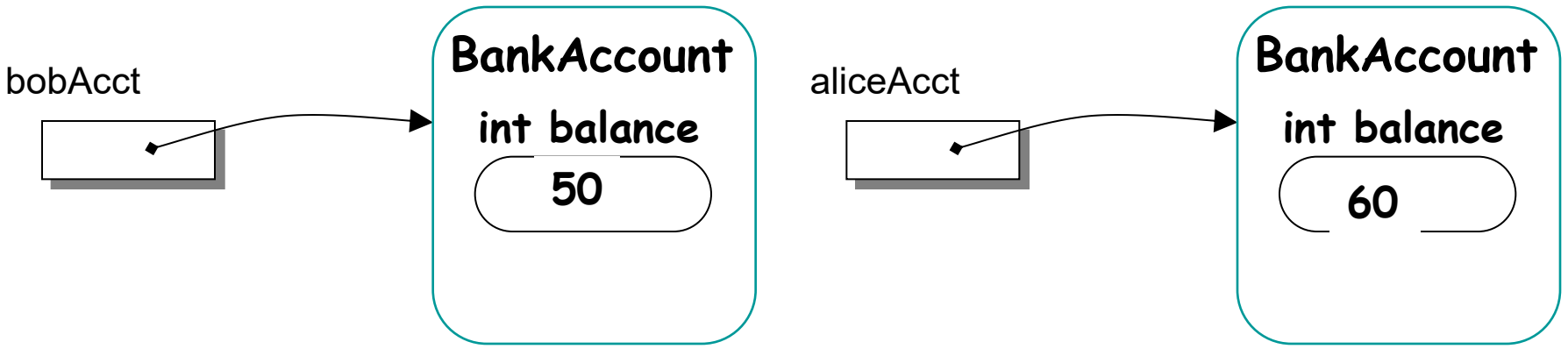
bobAcct = new BankAccount( );

BankAccount aliceAcct;

aliceAcct = new BankAccount();

bobAcct.deposit( 50 );

aliceAcct.deposit( 60 );
```



Example: Two Accounts

```
public class BankAccount
{
  private int balance;

  public void deposit( int amount )
  {
    this.balance = this.balance
                 + amount;
  }
  …
}
```

```
BankAccount bobAcct;

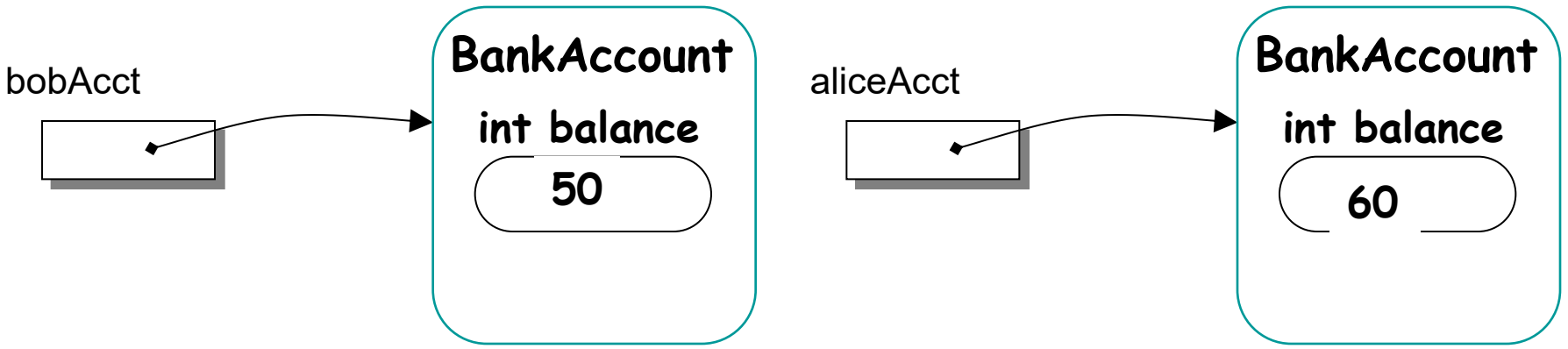
bobAcct = new BankAccount( );

BankAccount aliceAcct;

aliceAcct = new BankAccount();

bobAcct.deposit( 50 );

aliceAcct.deposit( 60 );
```



Using the “this” Keyword

```
public class Student {
   public Student() {
        transcript = new Transcript();
   }
   public Student(String s) {
        this.setSsn(s);
        transcript = new Transcript();
   }
   public Student(String s, String n, int i) {
        this.setSsn(s);
        this.setName(n);
        this.setAge(i);
        transcript = new Transcript();
   }
   // etc.
}
```

3.3 Method Overloading



Method Overloading





Overloading is a language mechanism that allows two or more different methods belonging to the same class to have the ***same*** name as long as they have ***different argument signatures***.

Method Overloading

- For example, the Student class may legitimately define the following five different print method headers:
  - `void print(String fileName) { ... // version #1`
  - `void print(int detailLevel) { ... // version #2`
  - `void print(int detailLevel, String fileName) { … // version #3`
  - `int print(String reportTitle, int maxPages) { ... // version #4`
  - `boolean print() { ... // version #5`

  that the print method is said to be **overloaded**.

Method Overloading

- Note that all five of the methods differ in terms of their argument signatures:
  - `#1 (String)`
  - `#2 (int)`
  - `#3 (int, String)`
  - `#4 (String, int)`
  - `#5 ()`

Method Overloading

- then choose which of these five "flavors" of print method we'd like a Borrower object to perform based on what form of message we send to a Borrower object:

```
Student s = new Student();
/* Invoking the version of print that takes a
   single String argument.*/
s.print("output.rpt");
/*Invoking the version of print that takes a
   single int argument.*/
s.print(2);
/*Invoking the version that takes two
   arguments, an int followed by a String.*/
s.print(2, "output.rpt");
```

Method Overloading



- They are not overloading
  - `boolean print() { ... // version #5`
  - `void print() { ... // version #6`

  Because they have the same argument signature

Method Overloading

- Note that there is no such thing as ***attribute*** overloading; that is, if a class tries to declare two attributes with the same name

```
public class Student {

private String studentId;

private int studentId;

// etc.
```

- That is false

Overloading Constructor

- Just as we are permitted to overload methods in Java, we are permitted to overload constructors.
- we may write as many different constructors for a given class as we wish, as long as they have different argument signatures. For example,

Overloading Constructor

```
public class Student {
   private String name;
   private String ssn;
   private int age;
   public Student() {
        this.setSsn("?");
   }
   public Student(String s) {
        this.setSsn(s);
   }
   public Student(String s, String n, int i) {
        this.setSsn(s);
        this.setName(n);
        this.setAge(i);
   }
   // Other methods omitted from this example.}
```

Overloading Constructor

```
Student s1 = new Student();

Student s2 = new Student("123-45-6789");

Student s3 = new Student("987-65-4321",
  "John Smith", 21);
```

An Important Caveat Regarding the Default Constructor



- There is one very important caveat about default constructors in Java: if we declare ***any*** of our own constructors for a class, with ***any argument signature***, then the default parameterless constructor is ***not*** automatically provided.

3.4 Methods



Requesting Object Services

- One way to think of an object's methods are as ***services*** that can be requested of the object by the application as a whole
- In order to request some service of an ***object 'x'***, we need to know ***the specific language*** with which to communicate with x
  - We need to be clear as to exactly which of ***x's methods/services*** we want it to perform
  - Depending on the service request, we may need to give x ***some additional information*** so that x knows exactly how to proceed
  - x in turn needs to know whether we expect it to ***report back*** the outcome of what it has been asked to do

An Example

- With respect to household chores, a Person is capable of:
  - mowing the lawn
  - taking out the trash
  - washing the car
- We decide that we want our teenaged sons Larry, Moe, and Curley, to do each of these three chores

An Example, cont.

- How would we ask them to do this?
  - "Please *** wash the car."
  - "Please *** take out the trash."
  - "Please *** mow the lawn."

An Example, cont.

- How would we ask them to do this?
  - "Please wash the car (the Camry)."
  - "Please take out the trash."
  - "Please mow the lawn, and let me know if you see any crabgrass."

clarification

reporting back

Method Headers

- We take care of communicating these 3 aspects of each method by defining a ***method header***
- A formal specification of how that method is to be invoked -- it consists of:
  - A method's ***name***;
  - An optional list of ***arguments*** (names and types of variables) to be passed to the method, enclosed in parentheses; and
  - A method's ***return type*** – that is, the data type of the information that is going to be passed back, if any, when the method is finished executing

Method Headers, cont.

- Example of a Student method:

```
boolean registerForCourse(String
  courseId, int sectionNumber)
```

return type (can be `void` if no response is required)

method name

optional argument list (can be empty ( ))

Method Headers, cont.

- Other examples:

```
boolean isHonorsStudent()
int getAge()
void changeName(String newName)
```

return type	method name	(optional) argument list

- We declare a return type of `void` when we don't need any data to be passed back to us

Argument Signatures

- The term ***argument signature*** refers to the order, types, and number of arguments comprising a method signature, but not their specific names
- Method signature:

```
boolean registerForCourse(String courseId, int sectionNumber)
```



has argument signature: `(String, int)`

Method Bodies

- We must also program the internal details of how each method should behave when invoked – ***this represents business logic***

```
public class Student {
  float gpa;

  boolean isHonorsStudent() {
      // All attributes are "in scope" for all
    methods.
      if (gpa >= 3.5) return true;
      else return false;
      // "return" statement not needed for void
    methods
    }
```

3.5 Message Passing Between Objects



Message Passing and Dot Notation



- To represent in code that a service is being requested of an object, we invoke a method on that object's reference variable using ***dot notation***

```
Student x = new Student();

x.registerForCourse("MATH 101", 10);
```

← a "message"

- A period ("dot") separates the object reference variable from the method (function) call
- The object is the context for the method … think of the notation x. as "talking to object x"

Passing Messages



- Returning to our example of asking our teenaged sons to do various chores, we need to target each request for a particular son:
  - "**Larry**, please wash the car (the Camry)."
  - "**Moe**, please take out the trash."
  - "**Curley**, please mow the lawn, and let me know if you see any crabgrass."

Passing Messages

- Now, let's turn this into Java code:
  - Person is a class
  - We have defined the following methods for Person:

```
void washTheCar(Car c)
void takeOutTheTrash()
boolean mowTheLawn()
```

Passing Messages



- We have three Person objects/instances:

```
Person larry = new Person();
Person moe = new Person();
Person curley = new Person();
```

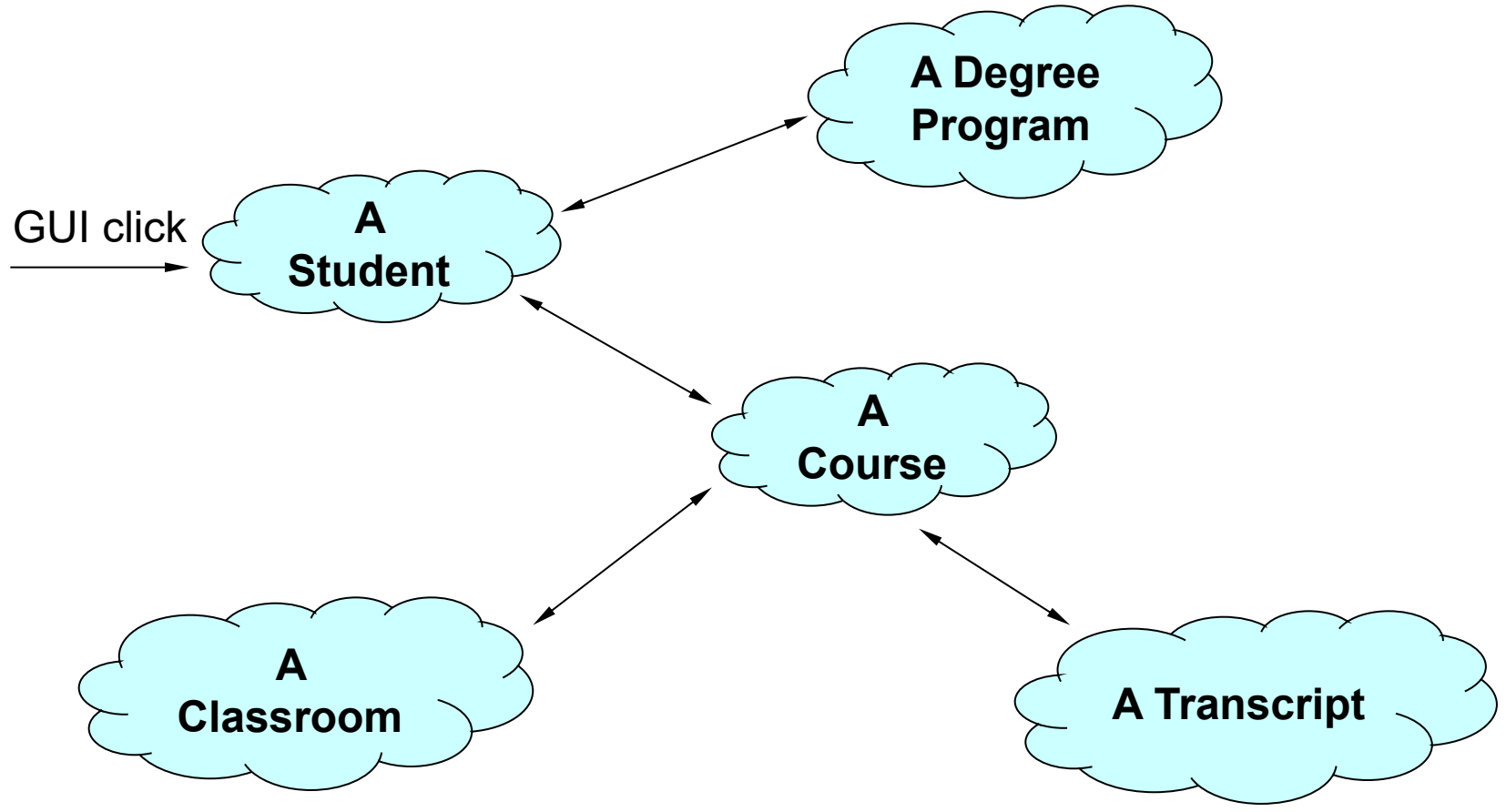
- While we're at it, let's define the Car class/abstraction:

```
Car camry = new Car();
```

- We send a message to each, indicating the service that we wish each of them to perform:

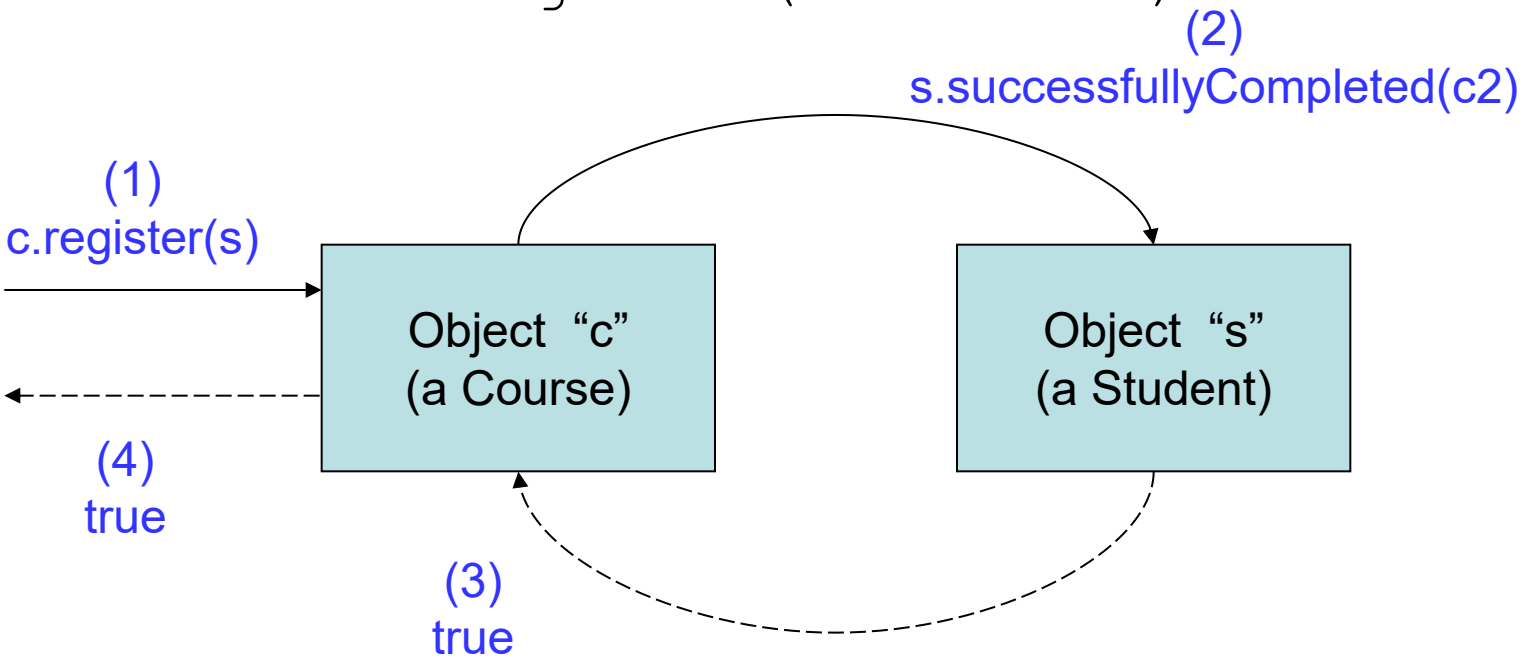
```
larry.washTheCar(camry);
moe.takeOutTheTrash();
curley.mowTheLawn();
```

Events Drive Object Collaboration



Message Passing Between Objects

- Class Student
  - `boolean successfullyCompleted(Course c)`
- Class Course
  - `boolean register(Student s)`



Obtaining Handles on Objects

- The ways of A handle on B
  - ***Object A might maintain a reference to B as one of A's attributes.***

```
public class Student {
// Attributes.
String name;
Professor facultyAdvisor;
// etc.
```

Obtaining Handles on Objects

- The ways of A handle on B
  - ***Object A may be handed a reference to B as an argument of one of A's methods.***

```
c.register(s);
```

Obtaining Handles on Objects

- The ways of A handle on B
  - ***A reference to object B may be made "globally available" to the entire application***
  - ***Object A may have to explicitly request a handle/reference to B by calling a method on some third object C.***

For example



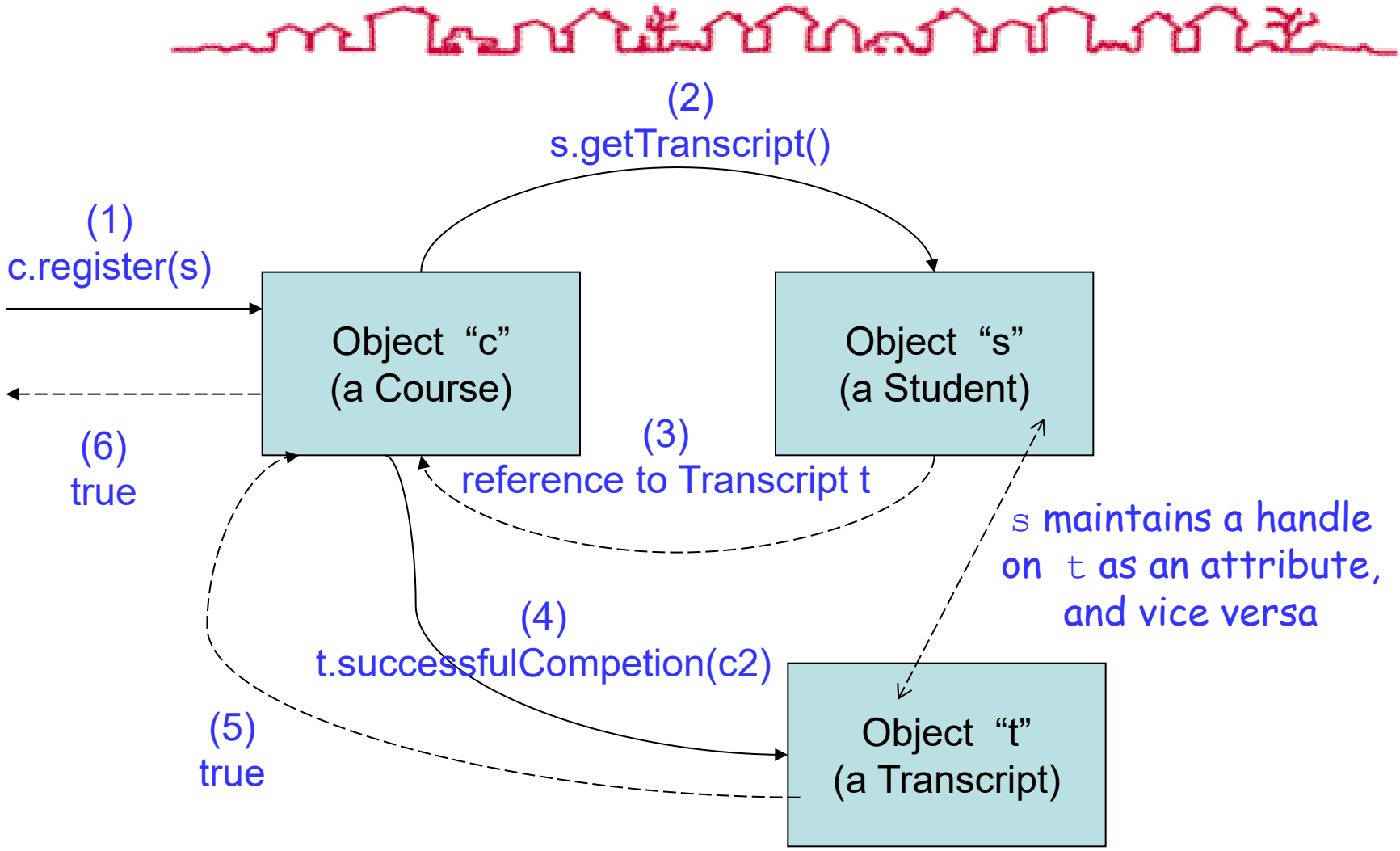
- First, we'll introduce a third object: a Transcript object **t**, which represents a record of all courses taken by Student objects.
- Furthermore, we'll assume that Student s maintains a handle on Transcript **t** as one of **s**'s attributes (specifically, the transcript attribute) and, conversely, that Transcript **t** maintains a handle on its "owner," Student **s**, as one of **t**'s attributes:

For example



```
public class Student {
// Attributes.
  Transcript transcript;
// etc.
}
public class Transcript {
// Attributes.
  Student owner;
// etc.
}
```

For example


(2)
s.getTranscript()
(1)
c.register(s)
Object "c"
(a Course)
Object "s"
(a Student)
(6)
true
(3)
reference to Transcript t
s maintains a handle
on t as an attribute,
and vice versa
(4)
t.successfulCompetion(c2)
(5)
true
Object "t"
(a Transcript)

3.6 Information Hiding/Accessibility



Information Hiding

- In a well designed OO application, a class **publicizes**
  - ***what* it can do** – i.e. the services it is capable of providing, or its method signatures

  but ***hides*** the internal details both of
  - ***how* it performs** these services (method bodies) and
  - **the *data* (attributes)** that it maintains in order to *support* these services
- Example: Television, Computer.

Visibility



- We use the term **visibility** to refer to whether or not a particular feature (attribute/method) can be accessed from client code by applying dot notation to the object's reference variable

Visibility, cont.



- **Public** visibility:

```
public class Student {
    public String name;
    // etc.
```

```
public class MyProgram {
    public static void main(String[]
args){
        Student x = new Student();
         x.name = "Fred Schnurd";  // OK!
        // etc.
```

Visibility, cont.

- **Private** visibility:

```
public class Student {
    public String name;
    private String ssn;
    // etc.
```

```
public class MyProgram {
    public static void main(String[] args){
        Student x = new Student();
        x.ssn = "123-45-6789"; // NOT OK TO MODIFY!
        System.out.println(x.ssn); // NOT OK TO
ACCESS!
```

"Variable ssn in class Student not accessible from class MyProgram"

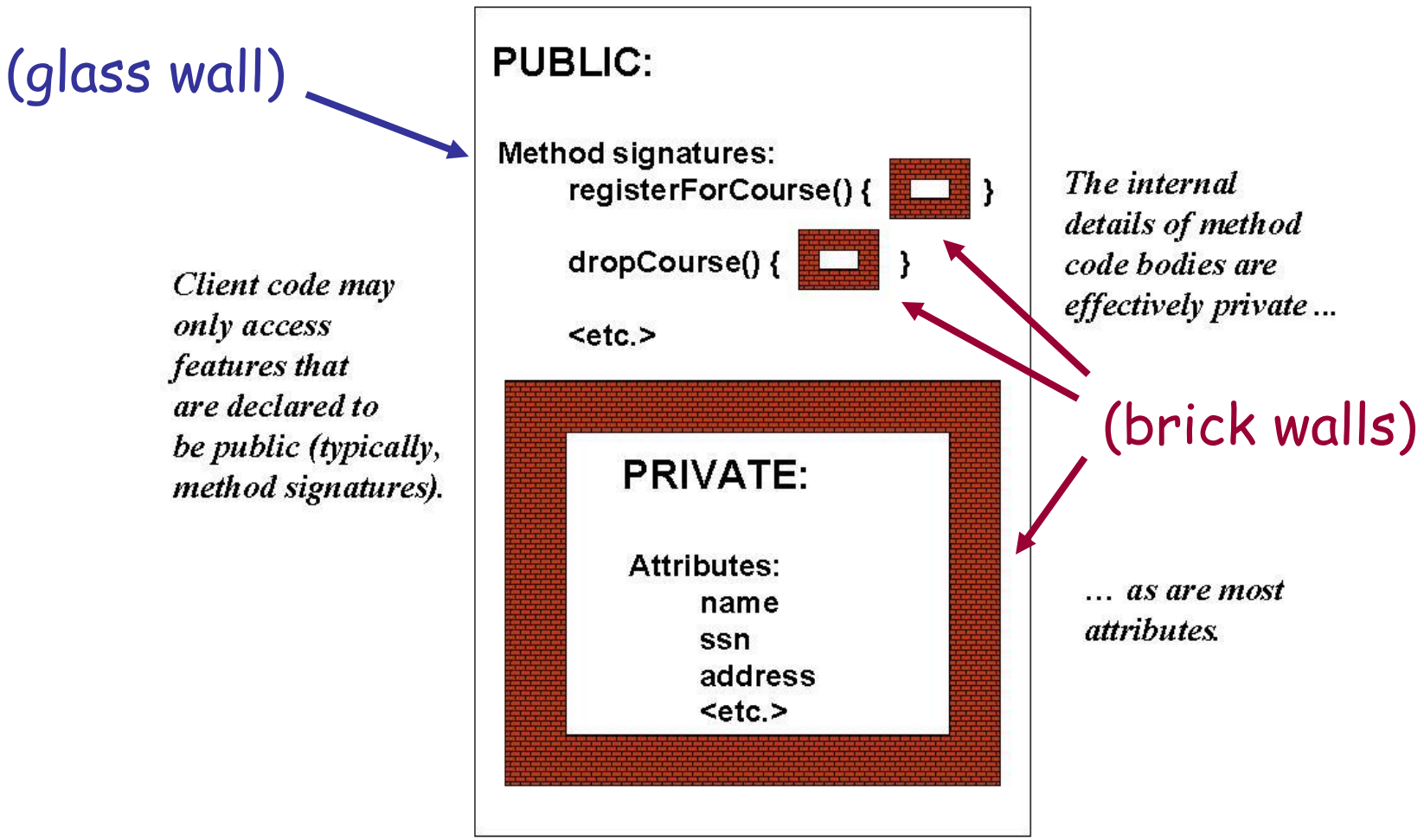
Visibility, cont.

- The same applies to methods:

```
public class Student {
    public String name;
    private String ssn;
    // etc.
   public void someMethod() { … }
   private void anotherMethod() { … }
```

```
public class MyProgram {
    public static void main(String[] args){
        Student x = new Student();
        x.someMethod(); // OK
        x.anotherMethod(); // NOT OK!
```

Info. Hiding, cont.



Encapsulation, Revisited



- It is useful to think of an object as a ‘fortress’ that ‘guards’ its data
  - We cannot directly access the values of an object’s ***privately***-declared attributes without an object’s permission and knowledge, i.e. …
  - We must use one of an object’s ***publicly*** accessible methods
    - Example: wallet access
- An object is responsible for ensuring the integrity of its own data

Info. Hiding Helps to Ensure Data Integrity



- As an example, let's say that we wish to store a Student's birthdate as a String attribute:

```
private String birthdate;
```

  - Our intention is to record birthdates in the format "mm/dd/yyyy"
  - We provide a Student method with signature:

```
boolean updateBirthdate(String d)
```

  - The single argument to the updateBirthdate() method as we've declared it is a String which can be of literally any format
  - How will we ensure that dates conform to our desired format?

Info. Hiding, cont.



- We'll provide logic within the method which verifies and, if necessary, rejects the request

```
public class Student {
   private String birthdate;
   // other details omitted

   public boolean updateBirthdate(String d) {
      if (date not in the format mm/dd/yyyy)
return false;
      else (if mm not in the range 01 to 12)
return false;
      // etc. for other validation tests
      else {
          // All is well!
          birthdate = d;  // or even:
setBirthdate(d);
          return true;
      }
   }
}
```

Info. Hiding Simplifies Code Maintenance

- Info. hiding insulates our application from changes to the hidden implementation details of a class
  - As an example, let's say that we design our Student class to have an attribute:

```
private int age;
```

  - and a get method as follows:

```
public int getAge() {
    return age;}
```

Info. Hiding, cont.



- We test and deploy our Student class, and it is used in dozens of applications throughout the University
- Later on, we decide that it was a silly idea to implement an age attribute -- we should have instead implemented a birthdate attribute, and compute the age whenever it is needed
- We change our class design as follows (see next slide):

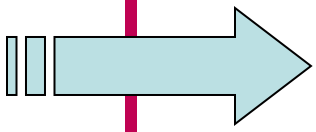
Info. Hiding, cont.

```
public class Student {
    // Explicit age attribute.
    private int age;

    public int getAge() {
       return age;

    }

    // etc.
```



```
public class Student {
    // NO age attribute!
    private String birthDate;

public int getAge() {
    return systemDate -
           birthDate;

// etc.
```

- The beauty is that, because all we've changed are ***private*** details of the class, ***we don't care which of these approaches is used!*** (cont.)

Info. Hiding, cont.



- With EITHER version of the Student class, the public method signature for getAge() is the same, and so client code won't have to change:

```
Student s = new Student();
// details omitted ...
int i = s.getAge();
```

- This method call worked before, when "age" was an explicit attribute; and it works now, when age is computed from the birthdate attribute
- Code changes made only to the private aspects of a class are said to be **encapsulated**

Info. Hiding, cont.

- Of course, all bets are off if the developer of a class changes one of its public method signatures
- For example, if we decided to change

```
int getAge()
```

to:

```
float getAge()
```

then the client code

```
int i = s.getAge();
```

would no longer compile, and we would have a "ripple effect" throughout our application(s)

Info. Hiding, cont.

- Of course, all bets are off if the developer of a class changes one of its public method signatures
- For example, if we decided to change

```
int getAge()
```

to:

```
float getAge()
```

then the client code

```
int i = (int) s.getAge();
// a cast is needed
```

Accessing One's Own Features

- Whether public or private, a class's features are visible/accessible within all of its own methods
  - This is because the attributes are "in scope" at the class level

```
public class Student {
        private String name;

        public void printInfo() {
       System.out.println(name);
       // etc.
    }
    // etc.
```

Summary



- How to formally specify method headers, the "language" with which services may be requested of an object, and how to formulate messages—using dot notation—to actually get an object to perform such services
- That multiple objects often have to collaborate in carrying out a particular system function, such as registering a student for a course
- That an object A can only communicate with another object B if A has a handle on B, and the various ways that such a handle/reference can be obtained

Summary



- How a special type of procedure called a ***constructor*** is specified and used to control what is to occur when we instantiate new objects
- How ***overloading*** enables a class to have multiple methods with the same name and/or multiple constructors as long as their argument signatures are different

Exercise



- Read textbook, chapter 4 Object Interactions, p95-p164.